Template Method Design Pattern

1. Classification, Intent & Exam Triggers

- **Category:** Behavioral Design Pattern
- **Intent:** Defines the skeleton (fixed step sequence) of an algorithm inside a method in an abstract superclass, while deferring the implementation of specific varying steps to subclasses without changing the algorithm's overall structure.
- **Primary Problem Solved (Algorithm Duplication & Workflow Drift):** Eliminates duplicate workflow code across multiple classes that execute the exact same sequence of operations but differ only in specific steps (Formula: Same Process + Different Steps = Template Method).
- **Core OOP Principles:** Applies Inheritance ('is-a' via extends) and Generalization to share common code, Abstraction (abstract class & protected abstract methods) to enforce varying steps, Method Overriding (@Override) & Polymorphism to execute subclass-specific steps at runtime, and Encapsulation (final template method + private common steps) to lock the algorithm structure.
- **How to Spot in Exam Questions:** 'A platform offers **different types** that must follow the exact same overall step-by-step process, while allowing each type to customize specific steps (such as assessment, evaluation, or shipment). Which pattern should be used?'

2. Pattern Structure, Participants & UML Relationships

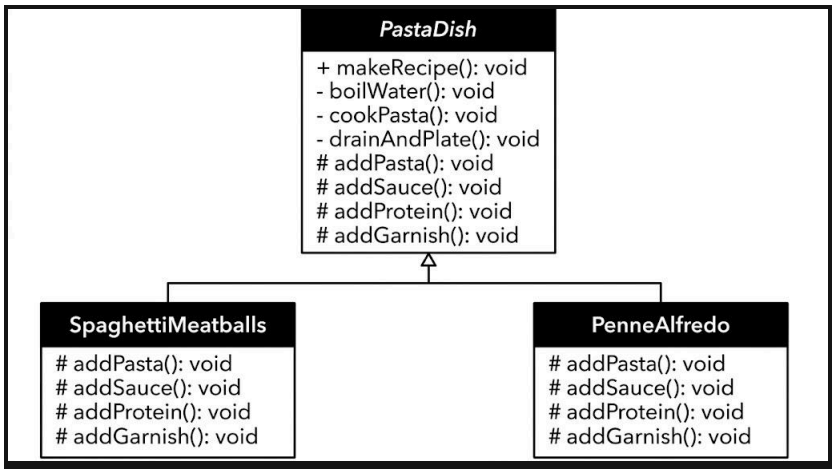
Participant Role	Responsibility in Pattern
1. AbstractClass (Abstract Superclass)	Declares the public final Template Method (locks the step sequence), implements Concrete/Common methods (invariant steps shared by all subclasses), and declares protected abstract Primitive Operations (varying steps subclasses must override).
2. ConcreteClass (Concrete Subclasses)	Extends AbstractClass (is-a) and overrides (@Override) the protected abstract primitive methods to provide domain-specific implementations for the variable steps without changing the workflow order.
3. Client (Main / Program)	Holds an AbstractClass reference pointing to a ConcreteClass instance (polymorphism) and invokes the single final template method to run the entire workflow.

Four Method Types Inside AbstractClass (High-Yield Exam Concept)

Method Type	Java Modifiers & UML Notation	Role & Exam Significance
1. Template Method	public final void (+ in UML)	Defines the master algorithm skeleton by calling steps in a fixed order (e.g., makeRecipe(), processData(), completeCourse()). Declared final so subclasses cannot override or reorder the sequence.
2. Common (Invariant) Steps	private void (- in UML) or protected void (# in UML)	Shared operations implemented once in AbstractClass (e.g., boilWater(), loadData(), validateRegistration(), generateCertificate()). Centralizes common code and prevents duplication.
3. Primitive (Variant) Steps	protected abstract void (# in italics / <>)	Placeholder steps with no body in AbstractClass (e.g., addPasta(), cleanData(), deliverAssessment(), evaluatePerformance()). Subclasses are forced to override these steps.
4. Hook Methods (Optional)	protected void (# in UML, concrete default/empty body)	Optional extension points with a default (often empty) implementation in AbstractClass. Subclasses may override them if needed, avoiding forced overrides when a step only applies to some subclasses.

UML Relationships

- **extends (Generalization / Inheritance — solid line with hollow triangle pointing to AbstractClass):** ConcreteClass subclasses inherit directly from AbstractClass. [Exam Fact: Unlike Decorator or Strategy which use Composition ('has-a'), Template Method relies on Inheritance ('is-a') and Method Overriding].
- **Why Abstract Class Instead of Interface? [L07 Exam Question]:** An Abstract Class is mandatory here because it provides both concrete methods (shared invariant steps + the final template method) and abstract methods (variant steps), and can hold instance state. An interface only defines a contract for unrelated classes.
- **Why Only Some Methods Are Abstract? [L07 BasicEngineering Q&A]:** In BasicEngineering, Math() and SoftSkills() are concrete because they are common to all engineering branches, whereas SpecialPaper() is abstract because it varies between Electronics and ComputerScience.
- **UML Visibility Symbols to Check in Diagrams:** '+' = public (Template Method), '#' = protected (abstract primitive steps overridden by subclasses), '-' = private (common superclass steps hidden from subclasses), Italics / <> = abstract class or abstract method.



3. Implementation Code Structure

```
// Step 1: Abstract Superclass with final Template Method, Common Steps & Abstract Steps
public abstract class PastaDish {

  // Template Method: final prevents subclasses from changing the sequence of steps
  public final void makeRecipe() {
     boilWater();     // Step 1: Common (Invariant)
     addPasta();      // Step 2: Varies (Abstract)
     cookPasta();     // Step 3: Common (Invariant)
     drainAndPlate(); // Step 4: Common (Invariant)
     addSauce();      // Step 5: Varies (Abstract)
     addProtein();    // Step 6: Varies (Abstract)
     addGarnish();    // Step 7: Varies (Abstract)
  }

  // Primitive (Abstract) Steps: protected so only subclasses can override them
  protected abstract void addPasta();
  protected abstract void addSauce();
  protected abstract void addProtein();
  protected abstract void addGarnish();

  // Common (Invariant) Steps: private so subclasses cannot alter shared behavior
  private void boilWater() {
     System.out.println("Boiling water");
  }
  private void cookPasta() {
     System.out.println("Cooking pasta");
  }
  private void drainAndPlate() {
     System.out.println("Draining and plating");
  }
}
```

```
// Step 2: Concrete Subclasses (implement ONLY the varying steps)
```

```
public class SpaghettiMeatballs extends PastaDish {
  @Override
  protected void addPasta() { System.out.println("Add spaghetti"); }
  @Override
  protected void addProtein() { System.out.println("Add meatballs"); }
  @Override
  protected void addSauce() { System.out.println("Add tomato sauce"); }
  @Override
  protected void addGarnish() { System.out.println("Add Parmesan cheese"); }
}

public class PenneAlfredo extends PastaDish {
  @Override
  protected void addPasta() { System.out.println("Add penne"); }
  @Override
  protected void addProtein() { System.out.println("Add chicken"); }
  @Override
  protected void addSauce() { System.out.println("Add Alfredo sauce"); }
  @Override
  protected void addGarnish() { System.out.println("Add parsley"); }
}

// Step 3: Client Execution (Polymorphism)
public class Program {
  public static void main(String[] args) {
    PastaDish dish1 = new SpaghettiMeatballs();
    dish1.makeRecipe(); // Runs fixed sequence with SpaghettiMeatballs steps

    PastaDish dish2 = new PenneAlfredo();
    dish2.makeRecipe(); // Runs fixed sequence with PenneAlfredo steps
  }
}
```

4. Key Comparisons, Trade-Offs & Real-World Challenges

Template Method vs. Strategy, Builder & Factory Method

Comparison Area	Template Method Pattern	Counterpart Pattern
vs. Builder (Sequence Control)	Superclass strictly enforces the sequence of steps in the final template method; client cannot reorder steps.	Builder: Client code (or Director) can dynamically direct the sequence and choose which construction steps to run.
vs. Factory Method (Primary Focus)	Behavioral: Focuses on how an algorithm is executed (fixed sequence of operational steps).	Factory Method (Creational): A specialized step focused on which object to instantiate.
Abstract Class vs. Interface	Abstract Class: Provides common base behavior (what a class is), can	Interface: Defines a contract (what a class must do) for unrelated

	hold state/fields, concrete + abstract methods, Single Inheritance.	classes, no instance variables (constants only), Multiple Inheritance.

Advantages, Disadvantages & Usage Rules

- **Key Advantages:**

  (1) Centralized Flow Control: The superclass controls the algorithm flow via a final template method; clients and subclasses cannot alter or corrupt the execution sequence.

  (2) Code Reusability & Zero Duplication (DRY): Common operations are implemented once in the abstract superclass, eliminating repeated boilerplate across subclasses.

  (3) Consistent Workflow & Easy Extension: All subclasses automatically follow the exact same process structure; adding a new variant only requires implementing the variable steps.

  (4) Framework Development (Hollywood Principle): Widely used in frameworks (JUnit, Spring, Servlets) where the framework controls the 'what' and 'when' and calls subclass methods for the 'how'.
- **When to Use:** When multiple classes share the exact same sequence of operations but differ in specific step implementations (Same Process + Different Steps), or when building reusable frameworks.
- **When NOT to Use:** When the client needs to direct or reorder the sequence of steps dynamically (use Builder), when the entire algorithm changes at runtime (use Strategy), or when the workflow sequence changes frequently.

Real-World Challenges

Challenge / Reflective Topic	How to Explain in the Exam
Sequence Inflexibility (Client Cannot Direct Steps)	Because the template method is final and locked in the superclass, client code cannot dynamically reorder or skip steps. If client-directed step sequencing is required, use the Builder Pattern instead (L07 Slide 15).
Accidental Method Hiding / Breaking Parent Logic	If parent methods are not restricted with final or private, a subclass might override a concrete method defined in the parent class, hiding the original definition and violating the Liskov Substitution Principle (L07 Slide 15).
Subclass Proliferation & Scattered Code	Every new workflow variation requires creating a new ConcreteClass subclass. Having many subclasses scatters step implementations across many files, making the system harder to navigate and maintain (L07 Slide 15).
Rigidity & Overgeneralization (Reflective Q2)	Occurs when the superclass forces an overly strict sequence or declares abstract steps that do not apply to certain subclasses. Avoid this by using Hook Methods (concrete methods with empty/default bodies) for optional steps or switching to Strategy.
Balancing Reusability & Customizability (Reflective Q1 & Q3)	Achieves reusability and consistency by locking the overall sequence in a final template method and centralizing common steps in the superclass, while achieving customizability by deferring variable steps to subclasses via protected abstract methods.